Predicting Titanic Survival: A SEMMA-Based Data Science Case Study

Yugm Maneshkumar Patel
San Jose State University
yugmmaneshkumar.patel@sjsu.edu

October 20, 2024

Abstract

This research paper explores the application of the SEMMA methodology to the Titanic dataset for understanding factors affecting passenger survival. Each step of the SEMMA process is explained in detail, accompanied by relevant Python code. This structured approach provides insights into data sampling, exploration, modification, modeling, and assessment. The findings from the logistic regression model highlight significant predictors of survival, which can aid future analyses and improve predictive accuracy.

1 Introduction

The SEMMA (Sample, Explore, Modify, Model, Assess) methodology is a structured approach for data mining that focuses on the entire data analysis process. This paper applies SEMMA to the Titanic dataset from Kaggle, analyzing the factors influencing passenger survival. The dataset consists of information about passengers, including their age, gender, ticket class, and more. The goal is to develop a model that predicts survival based on these features.

2 Methodology: SEMMA Process with Code

This section describes the SEMMA steps applied to the Titanic dataset, along with Python code snippets for each step.

2.1 Step 1: Sample

Goal: Select a representative sample of the data for modeling.

We load the Titanic dataset and take a random sample for our analysis.

```
import pandas as pd

# Load the dataset
data = pd.read_csv('train.csv')

# Take a random sample (optional)
sample_data = data.sample(frac=0.5, random_state=42)

# Display the first few rows of the sample
print(sample_data.head())
```

2.2 Step 2: Explore

Goal: Explore the data to identify patterns and data quality issues.

Next, we explore the dataset to understand its structure and identify any missing values.

```
import seaborn as sns
import matplotlib.pyplot as plt

# Check for missing values
print(sample_data.isnull().sum())

# Visualize the distribution of survival
```

```
plt.figure(figsize=(10,6))
sns.countplot(x='Survived', data=sample_data)
plt.title('Survival Distribution')
plt.show()
```

2.3 Step 3: Modify

Goal: Prepare and clean the data by handling missing values and feature engineering.

We clean the data by filling missing values and converting categorical variables.

```
# Fill missing Age values with the median
sample_data['Age'].fillna(sample_data['Age'].median(), inplace=
    True)

# Fill missing Embarked values with the mode
sample_data['Embarked'].fillna(sample_data['Embarked'].mode()[0],
     inplace=True)

# Convert categorical variables into numerical (One-Hot Encoding)
sample_data = pd.get_dummies(sample_data, columns=['Sex', '
    Embarked'], drop_first=True)

# Drop unnecessary columns
sample_data.drop(columns=['Cabin', 'Ticket', 'Name'], inplace=
    True)
```

2.4 Step 4: Model

Goal: Build a model to predict survival using a machine learning algorithm.

We build a logistic regression model to predict the likelihood of survival.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score

# Split the data into features (X) and target (y)
X = sample_data.drop(columns=['Survived'])
y = sample_data['Survived']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.3, random_state=42)

# Build a logistic regression model
model = LogisticRegression(max_iter=500)
model.fit(X_train, y_train)

# Predict on the test set
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy of the model: {accuracy:.2f}')
```

2.5 Step 5: Assess

Goal: Assess the performance of the model using evaluation metrics.

Finally, we evaluate the model's performance using the classification report and confusion matrix.

```
from sklearn.metrics import classification_report,
    confusion_matrix

# Classification report
print(classification_report(y_test, y_pred))
```

```
# Confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.show()
```

3 Results

The logistic regression model yielded an accuracy score of approximately 0.80, indicating a strong predictive capability. The classification report provided insights into precision, recall, and F1-score for each class.

4 Conclusion and Future Work

In this paper, we applied the SEMMA methodology to the Titanic dataset to analyze factors influencing passenger survival. The logistic regression model developed provides a baseline for future improvements. Future work could involve testing additional algorithms, feature selection, and hyperparameter tuning to enhance predictive accuracy.

5 References

1. Kaggle. Titanic: Machine Learning from Disaster. Retrieved from Kaggle

2. Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning: Data Mining, Inference, and Prediction. Springer.

3. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python, Journal of Machine Learning Research, 12, pp. 2825–2830.